

Neovim Complete Field Guide

Collin's setup • **Leader** = **Space**
Config checked: ~/.config/nvim
Searchable PDF edition

A practical map of the Vim language plus every explicit keybinding found in your Neovim configuration. Use **Ctrl+F** in the PDF for the command, key, action, or word you need.

BUILT-IN works in stock Vim/Neovim **YOUR MAP** explicitly configured in your setup **CONTEXT** only appears in an LSP, Markdown, Git, or dashboard buffer

1. Move

w word • **f** character • **gg** top



2. Select a thing

iw inner word • **a(** parentheses



3. Act on it

d delete • **c** change • **y** copy



4. Repeat / undo

. repeat • **u** undo • **Ctrl+r** redo

READ KEYS AS A SENTENCE

operator + motion

d w = delete to next word.

c i " = change inside quotes.

y a (= copy parentheses and contents.

The same motion becomes more useful when you pair it with **d**, **c**, **y**, **v**, **>**, or **<**.

MODE SURVIVAL KIT

i **Insert** before cursor.

a **Append** after cursor.

v V **Visual:** characters / whole lines.

Ctrl+v **Visual block:** rectangle / columns.

: **Command-line:** type an Ex command.

Esc **Normal mode:** safe home base.

DAILY 60-SECOND WORKFLOW

- Space f f** — open a file.
- Space f g** — search project text.
- /text** then **n** — search this file.
- ciw** — change a word; **.** to repeat.
- Space l f** — format code.
- :w** — save.
- Space l g** — open LazyGit.

Notation: **Space f f** means press Space, then f, then f—do not type the word "Space."

Scope: essential Vim/Neovim commands + all explicit mappings in the current configuration. Plugin defaults that you did not configure are intentionally excluded.

Move precisely, then act

Normal mode • motions work after d / c / y / v

CHARACTER, LINE & SCREEN

h j k l	Left, down, up, right.
0 / ^ / \$	Start of line / first non-space / end.
g_	Last non-blank character.
- / +	First non-blank of previous / next line.
Enter	First non-blank of next line.
H M L	Top / middle / bottom visible screen line.
Ctrl+u / Ctrl+d	Half-page up / down.
Ctrl+b / Ctrl+f	Full page back / forward.
zt zz zb	Put cursor line at top / center / bottom.

WORDS, PARAGRAPHS & FILE

w b e	Next word start / previous word start / word end.
W B E	Same, but whitespace-separated WORDS.
ge / gE	End of previous word / WORD.
{ }	Previous / next paragraph.
()	Previous / next sentence.
gg / G	First / last line of file.
:42 / 42G	Go to line 42.
%	Jump between matching (), [], {}, or HTML tag.
Ctrl+o / Ctrl+i	Older / newer jump location.

FIND ON ONE LINE

f x / F x	Find character x forward / backward; land on it.
t x / T x	Move until just before x, forward / backward.
; / ,	Repeat latest f/F/t/T in same / opposite direction.
* / #	Search word under cursor forward / backward.
g* / g#	Like * / #, but match inside larger words too.

Memory hook: find lands *on* it; till stops *before* it.

COUNTS = REPEAT THE NEXT COMMAND

3w	move 3 words	5dd	delete 5 lines	d2j	delete current + next 2 lines	2ci(	change inside next 2 parenthesized areas
-----------	--------------	------------	----------------	------------	-------------------------------	-------------	--

Edit with operators, text objects & registers

Normal / Visual / Insert modes

OPERATORS: THE VERBS

d	Delete (also copies to a register).
c	Change: delete then enter Insert mode.
y	Yank: copy to a register.
> / <	Indent right / left. Add = to auto-indent.
g~	Toggle case; gu lower; gU upper.
!	Filter selected text through a shell command.
=	Auto-format indentation using filetype rules.

Fast line forms: **dd** delete line • **cc** replace line • **yy** copy line • **>>** indent line.

TEXT OBJECTS: THE NOUNS

iw / aw	Inner word / a word including surrounding space.
i" / a"	Inside quotes / quotes plus contents.
i(i[i{	Inside parentheses / brackets / braces.
a(a[a{	Object including its delimiters.
it / at	Inside / around HTML/XML tag.
ip / ap	Inner paragraph / paragraph plus blank space.
is / as	Inner sentence / sentence plus space.

High-value combos: **ciw**, **di(**, **ya{**, **vit**.

INSERT, PASTE & REPEAT

i I	Insert at cursor / first non-blank on line.
a A	Append after cursor / end of line.
o O	Open line below / above and insert.
s S	Change character / entire line.
r x / R	Replace one character / overwrite mode.
x / X	Delete character under / before cursor.
p / P	Paste after / before cursor or line.
u / Ctrl+r	Undo / redo.
.	Repeat the last change—extremely useful.

REGISTERS: WHERE COPIED TEXT LIVES

TYPE	MEANING	EXAMPLE
"0	Latest yank	"0p paste last copied text
"+	System clipboard in your setup	"+y copy to OS clipboard
"a	Named register	"ayy store line; "ap paste
"_	Black-hole register	"_dd delete without replacing clipboard
: reg	Show registers	Inspect what was copied

Your config uses **unnamedplus**, so normal yank/delete/paste already shares the system clipboard.

VISUAL MODE & BLOCK EDITS

v / V	Select characters / whole lines, then use any operator.
Ctrl+v	Select a rectangle: move, then I to insert on every line, type, then Esc.
gv	Reselect the previous visual selection.
~	Swap selected text's case.
J	Join selected lines (or current + next line in Normal).
gJ	Join without inserting a space.

Find, replace, save & get unstuck

Search is local • Telescope searches projects

SEARCH WITHIN THE CURRENT FILE

KEY / COMMAND	PLAIN-ENGLISH ACTION
/text Enter	Search forward for text.
?text Enter	Search backward for text.
n / N	Next match / previous match.
:noh	Clear the highlighting, not the search pattern. YOUR MAP Space n h does this.
/\cword	Force case-insensitive search.
/\Cword	Force case-sensitive search.
/\Vliteral	Very literal: search punctuation without escaping it.
q/	Open searchable/editable search history.

Your setup uses **ignorecase + smartcase**: lowercase searches ignore case; adding any capital makes that search case-sensitive.

SUBSTITUTE: FIND & REPLACE

COMMAND	MEANING
:s/old/new/	Replace first “old” on current line.
:s/old/new/g	Replace every “old” on current line.
:%s/old/new/g	Replace every match in whole file.
:%s/old/new/gc	Whole file, ask yes/no for every replacement.
:10,25s/old/new/g	Only lines 10 through 25.
: '<,'>s/old/new/g	Only the current visual selection (press : while selected).
& / :&	Repeat latest substitute on this line.
:%&	Repeat latest substitute over whole file.

Flags: **g** = all per line; **c** = confirm; **i** = ignore case; **I** = match case.

FILES & QUITTING

<code>:w</code>	Write (save).
<code>:w new.md</code>	Save as a new name.
<code>:q</code>	Quit current window.
<code>:wq</code> / <code>:x</code>	Save then quit. <code>:x</code> only writes if changed.
<code>:q!</code>	Quit without saving current changes.
<code>:qa</code> / <code>:qa!</code>	Quit all / force quit all.
<code>:e file</code>	Edit / open file.

HELP & INSPECT

<code>:help keyword</code>	Open built-in help on a topic.
<code>K</code>	Keyword help under cursor (when supported).
<code>:map</code>	List mappings; use <code>:nmap</code> , <code>:imap</code> , etc. for a mode.
<code>:set option?</code>	Show one option, e.g. <code>:set number?</code> .
<code>:checkhealth</code>	Diagnose Neovim, providers, and plugins.
<code>:messages</code>	Show recent notifications/errors.

MARKS, MACROS & SPELLING

<code>m a</code> / <code>'a</code>	Set mark a / jump to its line.
<code>`a</code>	Jump to exact line + column of mark a.
<code>q a ... q</code>	Record actions into macro a.
<code>@a</code> / <code>@@</code>	Run macro a / run last macro again.
<code>]s</code> / <code>[s</code>	Next / previous spelling error.
<code>z=</code>	Spelling suggestions; <code>zg</code> add word.

Buffers, windows, tabs & lists

Think: buffer = file in memory • window = view

BUFFERS: OPEN FILES IN MEMORY

<code>:ls</code> / <code>:buffers</code>	List open buffers.
<code>:b name</code>	Switch to buffer by partial name.
<code>:bnext</code> / <code>:bprev</code>	Next / previous buffer.
<code>:bdelete</code>	Close buffer without necessarily closing window.
<code>Space b p</code> YOUR MAP	Visually pick any buffer.
<code>Space b d</code> YOUR MAP	Close current buffer.
<code>H</code> / <code>L</code> YOUR MAP	Previous / next buffer.

WINDOWS: SPLIT YOUR VIEW

<code>:split</code> / <code>:sp</code>	Horizontal split.
<code>:vsplit</code> / <code>:vs</code>	Vertical split.
<code>Ctrl+w h/j/k/l</code>	Move focus left / down / up / right.
<code>Ctrl+w w</code>	Cycle window focus.
<code>Ctrl+w c</code>	Close current window.
<code>Ctrl+w o</code>	Keep only current window.
<code>Ctrl+w =</code>	Make split sizes equal.
<code>Ctrl+w _</code> / <code>Ctrl+w </code>	Maximize height / width.

TABS, FOLDS & LISTS

<code>:tabnew</code>	New tab page.
<code>gt</code> / <code>gT</code>	Next / previous tab.
<code>za</code>	Toggle fold at cursor.
<code>zR</code> / <code>zM</code>	Open all / close all folds.
<code>:copen</code>	Open quickfix list (project-wide results).
<code>:cnext</code> / <code>:cprev</code>	Next / previous quickfix item.
<code>:lopen</code>	Open location list (window-local results).

WHEN TO USE WHICH FINDER IN YOUR SETUP

<code>Space f f</code>	<code>Space f g</code>	<code>/</code>	<code>Space f /</code>
File finder: locate/open a file by name.	Live grep: search text across the project.	Vim search: find text in the current file.	Fuzzy finder: filter the current buffer by characters in order.

Find files, explore projects & manage sessions

All are Normal mode • Leader = Space

YOUR MAP FIND & TELESCOPE

KEY	ACTION	USE IT WHEN...
Space f f	Find files	you know part of a file name.
Space f g	Find text (live grep)	you need a phrase anywhere in the project.
Space f w	Find word under cursor	you want references to the word you are on.
Space f b	Find buffers	you want an already-open file.
Space f r	Find recent files	you opened it recently but forgot its location.
Space f h	Find help	you need Vim/Neovim docs.
Space f k	Find keymaps	you forgot a mapping—best first stop.
Space f d	Find diagnostics	you need an error/warning in the project.
Space f /	Find in current buffer	you want fuzzy, in-file filtering.
Space f R	Resume last picker	you accidentally closed a finder.
Space f p	Find project	you want to switch projects.

YOUR MAP FILE EXPLORER & BUFFERS

KEY	ACTION	USE IT WHEN...
Space n t	Toggle NvimTree	you want the file explorer.
Space n n	Focus NvimTree	the explorer is already visible.
Space n f	Reveal current file	you need to see where this file lives.
Space n r	Refresh tree	external file changes are missing.
Space n c	Collapse tree	you need a clean explorer.
Space n e	Expand tree	you want to inspect everything.
Space b p	Pick buffer	many tabs/buffers are open.
Space b d	Close buffer	you are done with current file.
H / L	Previous / next buffer	you are cycling open files.

SESSIONS & DASHBOARD

Space w r

Find session. Restore a saved workspace.

Space w s

Save session. Persist current layout/buffers.

Space ?

Buffer-local keymaps. Shows context keys for this buffer.

Sessions save buffers, layout, current directory, terminals, folds, tab pages, and local options in your setup.

DASHBOARD-ONLY KEYS

These work on the Alpha start screen—not in a normal editing buffer.

e

New empty file.

ff fr fs fp

Find files / recent / search text / projects.

nt ss lg fk

Explorer / sessions / LazyGit / keymaps.

p / q

Plugin manager / quit all.

PERSONAL DISPLAY & UI

F2

Toggle absolute line numbers in current window.

F3

Toggle relative line numbers in current window.

Space n h

Clear search highlighting.

Space u i

Toggle indent guides.

Space u s

Toggle indent scope highlight.

Space u a

Toggle automatic brackets/quotes.

Space u w

Toggle Git word-level diff.

Git, diagnostics, LSP & code completion

Some keys require the noted context

YOUR MAP GIT SIGNS: LINE-LEVEL CHANGES

KEY	ACTION	CONTEXT
]h / [h	Next / previous changed Git hunk	Git-tracked file
Space h s	Stage current hunk	Normal mode
Space h r	Reset current hunk	Normal mode
Space h s / Space h r	Stage / reset selection	Visual mode
Space h S / h R / h U	Stage file / reset file / unstage file	Normal mode
Space h p / h i	Preview hunk / inline preview	Normal mode
Space h b / h B	Blame current line / toggle line blame	Normal mode
Space h d / h D	Diff against index / against HEAD	Normal mode
Space h q / h Q / h L	Hunks to quickfix / all repo hunks / location list	Normal mode
i h	Select a Git hunk as a text object	Visual or operator-pending

YOUR MAP LAZYGIT & GIT HISTORY

KEY	ACTION
Space l g	Open LazyGit dashboard for repository Git workflow.
Space l G	Open LazyGit focused on current file.
Space l L	Open repository log in LazyGit.
Space l l	Open current file's log in LazyGit.
Space g c	Telescope Git commit history.
Space g b	Telescope Git branches.
Space g s	Telescope Git working-tree status.

Rule of thumb: use **Space h ...** for one hunk, **Space g ...** to find Git information, and **Space l g** for a full Git UI.

YOUR MAP **DIAGNOSTICS & LSP**

[d] / [d] Previous / next diagnostic; opens its message.

Space l d Show full diagnostic at cursor.

Space l f Format current LSP buffer. In Markdown, uses Prettier if available.

Space l i Toggle inlay hints when the attached language server supports them.

Ctrl+s Show function signature help in Insert or Select mode.

Diagnostic: an editor-reported issue, such as an error, warning, or hint. **LSP:** the language-aware engine that supplies diagnostics, formatting, completion, navigation, and more.

COMPLETION & SNIPPETS

Tab If a snippet is active, jump to its next placeholder; otherwise insert a normal tab.

Shift+Tab If a snippet is active, jump back to its previous placeholder; otherwise normal Shift+Tab.

Ctrl+n / **Ctrl+p** Built-in Insert-mode next / previous completion candidate.

Ctrl+x Ctrl+o Built-in omnifunc completion where supported.

Ctrl+y Accept selected built-in completion entry.

Your LSP enables automatic completion when a server supports it; completion menu behavior is configured to avoid automatically selecting the first result.

Markdown, TODOs & writing tools

These appear in Markdown / R Markdown buffers

MARKDOWN CONTEXT PREVIEW, LINKS & STRUCTURE

KEY	ACTION
Space m p	Toggle rendered Markdown preview.
Space m h	Toggle hybrid preview (editing plus rendered elements).
Space m s	Toggle split preview.
Space m l	Follow link under cursor.
Space m n / Space m N	Next / previous Markdown link.
Space m t	Format Markdown table.
Space m c	Toggle Markdown task checkbox. Works in Normal and Visual mode.
Space m f / Space m F	Fold / unfold current Markdown section.

MARKDOWN CONTEXT PROJECT WRITING HYGIENE

KEY	ACTION
Space m w	Add misspelled word at cursor to project dictionary.
Space m W	Add all identified misspellings to project dictionary.
Space m a	Toggle Markdown format-on-save.
Space m T	Toggle table mode: disables wrap/linebreak so tables are easier to edit.
Space m d	Run Markdown linter now.
Enter	Follow/create Markdown links through mkdnflow in Normal, Insert, or Visual mode.
]s / [s	Next / previous spelling error (built-in spell checking).

Markdown buffers automatically enable US English spell checking, soft wrapping, line breaks, and break indentation in your setup.

YOUR MAP TODO COMMENTS

]t / **[t** Next / previous TODO-style comment in code.

Space f t Search TODO comments through Telescope.

Space f T Send TODO comments to the quickfix list.

Space f L Send TODO comments to the location list.

Use clear code comments like TODO, FIX, FIXME, HACK, WARN, PERF, NOTE, or TEST to make them discoverable.

AUTOPAIRS & EDITING EXPECTATIONS

Space u a Turn automatic matching parentheses, brackets, quotes, etc. on/off.

Alt+e **Fast wrap:** in Insert mode, wrap the current text with a paired delimiter when autopairs recognizes the context.

Backspace With autopairs on, delete paired delimiters together when appropriate.

Enter With autopairs on, creates a well-indented line inside matching pairs where appropriate.

Autopairs is disabled in Telescope prompts, Spectre, Snacks picker input, and Git commit messages to keep those tools predictable.

Find the command by what you want to do

Search this PDF for exact key strings or ordinary words

Search tips: Press **Ctrl+F** in your PDF viewer and search “replace”, “Git”, “Markdown”, “diagnostic”, “window”, “buffer”, “format”, or the literal mapping such as “Space h s”.

dd — delete without replacing clipboard

*** / #** — search word under cursor forward / backward

. — repeat last change

/ text — search current file forward

:bdelete — close buffer

:buffers — list buffers

:checkhealth — diagnose Neovim

:copen — open quickfix list

:e file — open file

:help — built-in documentation

:ls — list buffers

:messages — show messages/errors

:noh — clear search highlight

:q / :q! — quit / force quit

:qa — quit all

:reg — show copied registers

:set option? — inspect option

:sp / :vs — horizontal / vertical split

:tabnew — new tab

:w / :wq / :x — save / save+quit / save-if-changed+quit

0 ^ \$ — start / first text / end of line

Ctrl+b / Ctrl+f — full page up / down

Ctrl+d / Ctrl+u — half page down / up

Ctrl+o / Ctrl+i — older / newer jump

Ctrl+r — redo

Ctrl+v — visual block selection

Ctrl+w h/j/k/l — move between split windows

Esc — return to Normal mode

F2 / F3 — toggle absolute / relative numbers

H / L — previous / next buffer in your setup

i / a / o — insert before / append / new line below

iw / aw — inner word / word plus space

n / N — next / previous search match

p / P — paste after / before

q a ... q — record macro a

u — undo

v / V — visual character / line selection

w / b / e — word next / previous / end

x — delete character

yy / dd / cc — copy / delete / change line

za / zR / zM — toggle / open all / close all folds

Space ? — buffer-local mappings

Space b p / b d — pick / close buffer

Space f f — find files

Space f g — project text search

Space f k — find keymaps

Space f R — resume finder

Space g c / b / s — Git commits / branches / status

Space h s / h r — stage / reset Git hunk

Space l d / l f / l i — diagnostic / format / inlay hints

Space l g — LazyGit

Space m p — Markdown preview

Space m c — toggle Markdown checkbox

Space m d — lint Markdown

Space n t — toggle file explorer

Space n f — reveal current file in explorer

Space n h — clear search highlight

Space u a — toggle autopairs

Space u i — toggle indent guides

Space w r / w s — find / save session

[d /]d — previous / next diagnostic

[h /]h — previous / next Git hunk

[t /]t — previous / next TODO comment

WHAT IS INTENTIONALLY NOT HERE?

Vim has hundreds of historic, niche, and filetype-dependent commands. This guide includes the high-leverage core you are likely to need, plus **every explicit mapping in your current configuration**—including file explorer, Telescope, Git, LSP, Markdown, sessions, TODOs, autopairs, dashboard, and display controls.

HOW TO DISCOVER ANYTHING ELSE

`Space f k` opens a searchable view of your active mappings.
`:help keyword` explains any built-in. `:map`, `:nmap`, and `:imap` inspect mappings directly. `:checkhealth` diagnoses setup issues.

Extension Mapping Atlas

All current user-facing mappings extracted from your live Neovim setup: global maps, NvimTree, and a loaded Markdown buffer. Internal `<Plug>` implementation keys are omitted because they are not keys you type.

Search it: Ctrl+F NvimTree, Markdown, Gitsigns, Telescope, Space h, gx, grn, gcc, or an exact key.

Every active global mapping

Global configuration + extension mappings

Includes your custom leader mappings, Telescope, Gitsigns, LSP, diagnostics, Treesitter, Matchit, comments, TODOs, buffers, and active Insert/Visual maps.

Key	Definition / action	Mode
Space ?	Buffer Local Keymaps (which-key)	Normal
Space gs	Git status	Normal
Space gc	Git commits	Normal
Space fp	Find project	Normal
Space fR	Resume last picker	Normal
Space f/	Find in buffer	Normal
Space fd	Find diagnostics	Normal
Space fk	Find keymaps	Normal
Space fh	Find help	Normal
Space fr	Find recent files	Normal
Space fb	Find buffers	Normal
Space fw	Find word under cursor	Normal
Space fg	Find text	Normal
Space ff	Find files	Normal
Space gb	Git branches	Normal
Space wr	Find session	Normal
Space ws	Save session	Normal
Space nt	Toggle nvim-tree	Normal
Space ne	Expand nvim-tree	Normal
Space nc	Collapse nvim-tree	Normal
Space nr	Refresh nvim-tree	Normal
Space nf	Reveal current file	Normal
Space nn	Focus nvim-tree	Normal
Space ui	Toggle indent guides	Normal
Space us	Toggle indent scope	Normal
Space lL	LazyGit repository log	Normal
Space lG	LazyGit current file	Normal

Key	Definition / action	Mode
Space lg	LazyGit dashboard	Normal
Space ll	LazyGit file log	Normal
Space hl	Hunks to location list	Normal
Space hQ	Repository hunks to quickfix	Normal
Space hq	Hunks to quickfix	Normal
Space hD	Diff against HEAD	Normal
Space hd	Diff against index	Normal
Space uw	Toggle Git word diff	Normal
Space hB	Toggle line blame	Normal
Space hb	Blame line	Normal
Space hi	Preview hunk inline	Normal
Space hp	Preview hunk	Normal
Space hU	Unstage buffer	Normal
Space hR	Reset buffer	Normal
Space hS	Stage buffer	Normal
Space hr	Reset hunk	Normal
Space hs	Stage hunk	Normal
Space bd	Close buffer	Normal
Space bp	Pick buffer	Normal
Space fL	Todo comments to location list	Normal
Space fT	Todo comments to quickfix	Normal
Space ft	Find todo comments	Normal
Space ua	Toggle autopairs	Normal
%	<Plug>(MatchitNormalForward)	Normal
H	Previous buffer	Normal
L	Next buffer	Normal
[%	<Plug>(MatchitNormalMultiBackward)	Normal
[h	Previous hunk	Normal
[	Add empty line above cursor	Normal
[B	:brewind	Normal
[b	:bprevious	Normal
[<C-T>	:ptprevious	Normal

Key	Definition / action	Mode
[T	:trewind	Normal
[t	Previous todo comment	Normal
[A	:rewind	Normal
[a	:previous	Normal
[<C-L>	:lfile	Normal
[L	:lrewind	Normal
[l	:lprevious	Normal
[<C-Q>	:cpfile	Normal
[Q	:crewind	Normal
[q	:cpprevious	Normal
[D	Jump to the first diagnostic in the current buffer	Normal
[d	Jump to the previous diagnostic in the current buffer	Normal
]%	<Plug>(MatchitNormalMultiForward)	Normal
]h	Next hunk	Normal
]	Add empty line below cursor	Normal
]B	:blast	Normal
]b	:bnext	Normal
]<C-T>	:ptnext	Normal
]T	:tlast	Normal
]t	Next todo comment	Normal
]A	:last	Normal
]a	:next	Normal
]<C-L>	:lnfile	Normal
]L	:llast	Normal
]l	:lnext	Normal
]<C-Q>	:cnfile	Normal
]Q	:clast	Normal
]q	:cnnext	Normal
]D	Jump to the last diagnostic in the current buffer	Normal
]d	Jump to the next diagnostic in the current buffer	Normal
g%	<Plug>(MatchitNormalBackward)	Normal
g0	vim.lsp.buf.document_symbol()	Normal

Key	Definition / action	Mode
grt	vim.lsp.buf.type_definition()	Normal
gri	vim.lsp.buf.implementation()	Normal
grr	vim.lsp.buf.references()	Normal
grx	vim.lsp.codelens.run()	Normal
gra	vim.lsp.buf.code_action()	Normal
grn	vim.lsp.buf.rename()	Normal
gcc	Toggle comment line	Normal
gc	Toggle comment	Normal
gx	Opens filepath or URI under cursor with the system handler (file explorer, web browser, ...)	Normal
<C-W><C-D>	Show diagnostics under the cursor	Normal
<C-W>d	Show diagnostics under the cursor	Normal
<Tab>	vim.snippet.jump if active, otherwise <Tab>	Visual
Space hr	Reset selection	Visual
Space hs	Stage selection	Visual
%	<Plug>(MatchitVisualForward)	Visual
[%	<Plug>(MatchitVisualMultiBackward)	Visual
[N	Select previous sibling node	Visual
[n	Select previous node	Visual
]%	<Plug>(MatchitVisualMultiForward)	Visual
]N	Select next sibling node	Visual
]n	Select next node	Visual
a%	<Plug>(MatchitVisualTextObject)	Visual
an	Select parent (outer) node	Visual
g%	<Plug>(MatchitVisualBackward)	Visual
gra	vim.lsp.buf.code_action()	Visual
gc	Toggle comment	Visual
gx	Opens filepath or URI under cursor with the system handler (file explorer, web browser, ...)	Visual
ih	Select hunk	Visual
in	Select child (inner) node	Visual
<S-Tab>	vim.snippet.jump if active, otherwise <S-Tab>	Visual
<C-S>	vim.lsp.buf.signature_help()	Visual
Space hr	Reset selection	Visual

Key	Definition / action	Mode
Space hs	Stage selection	Visual
%	<Plug>(MatchitVisualForward)	Visual
[%	<Plug>(MatchitVisualMultiBackward)	Visual
[N	Select previous sibling node	Visual
[n	Select previous node	Visual
]%	<Plug>(MatchitVisualMultiForward)	Visual
]N	Select next sibling node	Visual
]n	Select next node	Visual
a%	<Plug>(MatchitVisualTextObject)	Visual
an	Select parent (outer) node	Visual
g%	<Plug>(MatchitVisualBackward)	Visual
gra	vim.lsp.buf.code_action()	Visual
gc	Toggle comment	Visual
gx	Opens filepath or URI under cursor with the system handler (file explorer, web browser, ...)	Visual
ih	Select hunk	Visual
in	Select child (inner) node	Visual
%	<Plug>(MatchitOperationForward)	Operator
[%	<Plug>(MatchitOperationMultiBackward)	Operator
]%	<Plug>(MatchitOperationMultiForward)	Operator
an	Select parent (outer) node	Operator
g%	<Plug>(MatchitOperationBackward)	Operator
gc	Comment textobject	Operator
ih	Select hunk	Operator
in	Select child (inner) node	Operator
<S-Tab>	vim.snippet.jump if active, otherwise <S-Tab>	Insert
<C-S>	vim.lsp.buf.signature_help()	Insert
<Tab>	vim.snippet.jump if active, otherwise <Tab>	Insert
<Tab>	vim.snippet.jump if active, otherwise <Tab>	Select
<S-Tab>	vim.snippet.jump if active, otherwise <S-Tab>	Select
<C-S>	vim.lsp.buf.signature_help()	Select
Space lc	Toggle LSP auto-completion for all completion-capable clients on this buffer.	LSP buffer
<C-Space>	Manually trigger LSP completion when LSP completion is enabled.	LSP buffer • Insert

NvimTree complete map inventory

NvimTree maps

Only work while focus is inside the file explorer. Includes all user-facing default NvimTree actions currently active.

Key	Definition / action	Mode
<Tab>	nvim-tree: Open Preview	Normal
<CR>	nvim-tree: Open	Normal
-	nvim-tree: Up	Normal
.	nvim-tree: Run Command	Normal
<lt>	nvim-tree: Previous Sibling	Normal
>	nvim-tree: Next Sibling	Normal
B	nvim-tree: Toggle Filter: No Buffer	Normal
C	nvim-tree: Toggle Filter: Git Clean	Normal
D	nvim-tree: Trash	Normal
E	nvim-tree: Expand All	Normal
F	nvim-tree: Live Filter: Clear	Normal
H	nvim-tree: Toggle Filter: Dotfiles	Normal
I	nvim-tree: Toggle Filter: Git Ignored	Normal
J	nvim-tree: Last Sibling	Normal
K	nvim-tree: First Sibling	Normal
L	nvim-tree: Toggle Group Empty	Normal
M	nvim-tree: Toggle Filter: No Bookmark	Normal
O	nvim-tree: Open: No Window Picker	Normal
P	nvim-tree: Parent Directory	Normal
R	nvim-tree: Refresh	Normal
S	nvim-tree: Search	Normal
U	nvim-tree: Toggle Filter: Custom	Normal
W	nvim-tree: Collapse All	Normal
Y	nvim-tree: Copy Relative Path	Normal
[e	nvim-tree: Prev Diagnostic	Normal
[c	nvim-tree: Prev Git	Normal
]e	nvim-tree: Next Diagnostic	Normal

Key	Definition / action	Mode
l c	nvim-tree: Next Git	Normal
a	nvim-tree: Create File Or Directory	Normal
b m v	nvim-tree: Move Bookmarked	Normal
b t	nvim-tree: Trash Bookmarked	Normal
b d	nvim-tree: Delete Bookmarked	Normal
c	nvim-tree: Copy	Normal
d	nvim-tree: Delete	Normal
e	nvim-tree: Rename: Basename	Normal
f	nvim-tree: Live Filter: Start	Normal
g p	nvim-tree: Move	Normal
g e	nvim-tree: Copy Basename	Normal
g y	nvim-tree: Copy Absolute Path	Normal
g ?	nvim-tree: Help	Normal
m	nvim-tree: Toggle Bookmark	Normal
o	nvim-tree: Open	Normal
p	nvim-tree: Paste	Normal
q	nvim-tree: Close	Normal
r	nvim-tree: Rename	Normal
s	nvim-tree: Run System	Normal
u	nvim-tree: Rename: Full Path	Normal
x	nvim-tree: Cut	Normal
y	nvim-tree: Copy Name	Normal
<2-RightMouse>	nvim-tree: CD	Normal
<2-LeftMouse>	nvim-tree: Open	Normal
	nvim-tree: Delete	Normal
<BS>	nvim-tree: Close Directory	Normal
<C-X>	nvim-tree: Open: Horizontal Split	Normal
<C-V>	nvim-tree: Open: Vertical Split	Normal
<C-T>	nvim-tree: Open: New Tab	Normal
<C-R>	nvim-tree: Rename: Omit Filename	Normal
<C-K>	nvim-tree: Info	Normal
<C-E>	nvim-tree: Open: In Place	Normal

Key	Definition / action	Mode
<C-]>	nvim-tree: CD	Normal
D	nvim-tree: Trash	Visual
c	nvim-tree: Copy	Visual
d	nvim-tree: Delete	Visual
gy	nvim-tree: Copy Absolute Path	Visual
m	nvim-tree: Toggle Bookmark	Visual
x	nvim-tree: Cut	Visual
	nvim-tree: Delete	Visual

Markdown extension complete map inventory

Markdown maps

Only work in Markdown buffers. Includes your custom Markdown mappings plus Markview and Mkdntflow defaults.

Key	Definition / action	Mode
<Tab>	Jump to next link	Normal
<CR>	Follow link, toggle to-do, or create link from selection	Normal
Space md	Lint Markdown	Normal
Space mT	Toggle table mode	Normal
Space ma	Toggle format on save	Normal
Space mW	Add all project spelling words	Normal
Space mw	Add project spelling word	Normal
Space mF	Unfold section	Normal
Space mf	Fold section	Normal
Space mc	Toggle task	Normal
Space mt	Format table	Normal
Space mN	Previous link	Normal
Space mn	Next link	Normal
Space mL	Follow link	Normal
Space ms	Toggle preview split	Normal
Space mh	Toggle hybrid preview	Normal
Space mp	Toggle preview	Normal
O	Create list item above and enter insert mode	Normal
[[	Jump to previous heading	Normal
[]	<Cmd>:MkdntPrevHeadingSame<CR>	Normal
]]	Jump to next heading	Normal
][	<Cmd>:MkdntNextHeadingSame<CR>	Normal
gx	Tree-sitter based link opener from `markview.nvim`.	Normal
gO	Show an Outline of the current buffer	Normal
o	Create list item below and enter insert mode	Normal
<S-Tab>	Jump to previous link	Normal
<CR>	Follow link, toggle to-do, or create link from selection	Visual

Key	Definition / action	Mode
Space mc	Toggle task	Visual
[[	:<C-U>exe "normal! gv" call search('\%(^#\{1,5\}\s\+\s\ ^S.*\n^[=-]\+\\$)', "bsW")<CR>	Visual
]]	:<C-U>exe "normal! gv" call search('\%(^#\{1,5\}\s\+\s\ ^S.*\n^[=-]\+\\$)', "sW")<CR>	Visual
<S - Tab>	Jump to previous table cell	Insert
<M - CR>	Jump to previous table row	Insert
<S - CR>	Insert new line in table cell	Insert
<C - T>	Indent list item and update numbering	Insert
<C - D>	Dedent list item and update numbering	Insert
<Tab>	Jump to next table cell	Insert
<CR>	Follow link, toggle to-do, or create link from selection	Insert
>	Active mapping	Insert

Debug & test management

Collin's setup • Leader = Space
Generated 2026-09-03

DEBUG SESSION CONTROLS

Normal	Space d b	Toggle breakpoint
Normal	Space d B	Conditional breakpoint
Normal	Space d c	Continue / start debugger
Normal	Space d p	Pause debug session
Normal	Space d i	Step into
Normal	Space d o	Step over
Normal	Space d O	Step out
Normal	Space d r	Restart debug session
Normal	Space d q	Terminate debug session
Normal	Space d u	Toggle DAP UI
Normal	Space d v	Toggle DAP virtual text
Normal	Space d e	Evaluate expression

TEST MANAGEMENT

Normal	Space t n	Run nearest test
Normal	Space t f	Run current file
Normal	Space t a	Run test suite
Normal	Space t d	Debug nearest test
Normal	Space t s	Toggle test summary
Normal	Space t o	Show test output
Normal	Space t x	Stop test
Normal	Space t v	Fallback: run nearest test
Normal	Space t V	Fallback: run current file
Normal	Space t A	Fallback: run test suite

Adapters, runners & context

Commands are searchable • context is explicit

DEBUG ADAPTERS

- **Python:** debugpy; uses project `.venv/bin/python` when available.
- **JavaScript / TypeScript:** js-debug-adapter (Node launch or attach).
- **Go:** Delve.
- **Rust, C, C++:** CodeLLDB.
- **.NET:** netcoredbg.

TEST ADAPTERS

- Python / pytest
- Go, Rust, CTest, .NET
- Vitest and Jest (dependency-aware selection)
- vim-test fallback for other runners

CONTEXT & COMPLETION

Normal, LSP buffer

Space l c

Toggle automatic LSP completion.

Insert, LSP buffer

Ctrl-Space

Request completion manually when enabled.

Context matters. Debug keys require an installed adapter; test keys require a detected adapter/project; LSP completion keys appear only after a compatible server attaches.